Introduction au système Linux

- Système d'exploitation **multi-utilisateurs** (connexion simultanée de plusieurs utilisateurs) et **multi-tâches** (décomposition d'une application en plusieurs tâches qui s'exécutent simultanément).
- L'environnement standard de programmation est le **C** et plus récemment le **C++**.
- L'environnement réseau est construit sur les protocoles **TCP/IP**.

Philosophie de Linux

- Un ensemble de règles assurent le fonctionnement et le succès de Linux. Le fameux paradigme « tout est fichier ». Toutes les fonctionnalités du système d'exploitation sont accessibles via de simples fichiers : périphériques, processus, mémoire, etc.

- Fini le développement seul dans son coin, la moindre petite ébauche de votre projet doit être soumise à la communauté afin d'avertir un maximum de personnes pouvant intervenir de différente manière au niveau du développement, soit en soulevant les bugs que vous vous attacherez à corriger le plus rapidement possible.

La création de Linux

- En 1991, un étudiant finlandais nommé Linus Torvalds décide de créer un nouveau noyau basé sur Minix (un dérivé d'UNIX) afin de contourner ses limitations et celles de MS-DOS. Ce nouveau noyau est distribué gratuitement sur Internet en août 1991 dès sa version 0.1 sous le nom de « Linux ».

- Actuellement, le noyau de Linux est à la révision 6.12.71 (12/02/2026) et est librement téléchargeable.

Les systèmes Linux du marché

Ubuntu C'est une distribution commerciale orientée vers le grand public distribué gratuitement par Canonical, qui édite des versions stables tous les six mois (maintenues 9 mois).

Fedora est une distribution communautaire supervisée par Red Hat, basée sur le système de gestion de paquetages logiciels RPM C'est une distribution qui met l'accent sur la nouveauté, ce qui signifie que ses logiciels sont très fréquemment mis à jour.

openSUSE est une distribution communautaire destinée tant à un usage grand public qu'à un usage professionnel. C'est une distribution indépendante réputée pour ses outils de configurations et sa stabilité.

Linux sur votre ordinateur

- Installer Linux sur une partition de votre disque. **Double boot** Windows-Linux

- Utiliser une **clé usb bootable** Linux.

- **Virtualiser** Linux sur votre PC avec Virtuabox par exemple. Possibilité d'avoir Linux, windows et Mac OS en même temps.

- **WSL (Windows Subsystem Linux) :** Sous-système Windows pour Linux

- Sous **MacOs** : Zsh disponible directement.

Le *Shell*

- Sous Linux, on appelle Shell l'***interpréteur de commandes*** qui fait office d'interface entre l'utilisateur et le système d'exploitation. Les shells sont des interpréteurs : cela signifie que chaque commande saisie par l'utilisateur (ou lue à partir d'un fichier) est syntaxiquement vérifiée puis exécutée.

- Il existe plusieurs interpréteurs Shell sous Linux : Bourne again shell (bash), C shell, Korn shell, Z shell etc…

- En connaître un, permet d'accéder aisément aux autres.

- Tous les Shells peuvent cohabiter à l'intérieur d'un même système Linux. L'administrateur fixe le Shell initial de chaque utilisateur dans le fichier de définition des utilisateurs : **/etc/passwd**

- Un shell possède un ***environnement de travail*** et un ***langage de programmation***.

Shell Unix / PowerShell Windows

PowerShell (Cmdlet)	PowerShell (Alias)	Shell Unix	Description
Get-ChildItem	gci, dir, ls	ls, dir	Liste les fichiers / répertoires du répertoire (courant)
Get-Content	gc, type, cat	cat	Obtenir le contenu d'un fichier
Get-Command	gcm	help, which	Liste des commandes
Get-Help	help, man	man	Aide
Clear-Host	cls, clear	clear	Efface l'écran
Copy-Item	cpi, copy, cp	cp	Copier un ou plusieurs fichiers / l'arborescence complète
Move-Item	mi, move, mv	mv	Désplacer un fichier/ répertoire
Remove-Item	ri, del, erase, rmdir, rd, rm	rm, rmdir	Suprimer un fichier / répertoire
Rename-Item	rni, ren, mv	mv	Renomer un fichier/ répertoire
Get-Location	gl, pwd	pwd	Afficher le répertoire de travail courant

L’environnement de travail du *Shell*

- Le shell permet d’exécuter une commande en mode interactif ou bien par l'intermédiaire de fichiers de commandes (*scripts*).

- En mode interactif, le **bash** affiche à l’écran une *chaîne d’appel* (appelée également *prompt* ou *invite*), qui se termine par défaut par le caractère # suivi d’un caractère **espace** pour l’administrateur système (utilisateur **root**) et par le caractère **$** suivi d’un caractère **espace** pour les autres utilisateurs. Cette chaîne d’appel peut être relativement longue.

- Ex : retraint@marylin:/home/retraint$

L’environnement de travail du *Shell*

- Le prompt est constituée du nom de connexion de l’utilisateur (retraint), du nom de la machine sur laquelle l’utilisateur travaille (marylin) et du chemin absolu du répertoire courant de l’utilisateur (*/home/retraint*). Il indique que le shell attend que l’utilisateur saisisse une commande et la valide en appuyant sur la touche **entrée**. Bash exécute alors la commande puis réaffiche la chaîne d’appel.

- Si l’on souhaite raccourcir la chaîne d’appel, il suffit de modifier la valeur de la variable prédéfinie du shell **PS1** (*Prompt Shell 1*).

- Ex : retraint@marylin:/home/retraint$ **PS1='$ '** .

L'environnement de travail du *Shell*

- Le Shell **bash** permet :
  - le rappel des commandes précédentes (gestion de l'historique) ; cela évite de taper plusieurs fois la même commande
  - la modification en ligne du texte de la commande courante (ajout, retrait, remplacement de caractères)
  - la gestion des travaux lancés en arrière-plan (appelés *jobs*) ; ceux-ci peuvent être démarrés, stoppés ou repris suivant les besoins
  - l'initialisation adéquate de variables de configuration (chaîne d'appel de l'interpréteur, chemins de recherche par défaut) ou la création de raccourcis de commandes (commande **alias**).

Avantages du *Shell*

- C’est un langage interprété : les erreurs peuvent être facilement localisées et traitées. Des modifications de fonctionnalités sont facilement apportées à l’application sans qu’il soit nécessaire de recompiler et faire l’édition de liens de l’ensemble.

- Le shell manipule essentiellement des chaînes de caractères: Ceci a pour avantage d’éviter des erreurs de typage.

- Le langage est adapté au prototypage rapide d’applications : les tubes, les substitutions de commandes et de variables favorisent la construction d’une application par assemblage de commandes préexistantes dans l’environnement Unix.

- Le shell permet de connecter des composants écrits dans des langages différents.

Inconvénients du *Shell*

- Système d'exploitation écrit à l'origine par des développeurs pour des développeurs, les shells utilisent une syntaxe d'accès difficile pour le débutant.
- L'oubli ou l'ajout d'un caractère **espace** provoque facilement une erreur de syntaxe.
- Le **bash** possède plusieurs syntaxes pour implanter la même fonctionnalité.
- Certains caractères spéciaux, comme les parenthèses, ont des significations différentes suivant le contexte : les parenthèses peuvent introduire une liste de commandes, une définition de fonction ou bien imposer un ordre d'évaluation d'une expression arithmétique.

Syntaxe d’une ligne de commande

- Syntaxe des commandes
  - **nom_commande [options] [arguments]**
  - C'est une suite de mots séparés par un ou plusieurs blancs. On appelle blanc un caractère **tab** (tabulation horizontale) ou un caractère **espace**.
  - Un mot est une suite de caractères non blancs.
  - Une option est généralement introduite par le caractère **tiret** (ex : **-a**) ou dans la syntaxe par deux caractères **tiret** consécutifs (ex : **--version**).
  - Les arguments désignent les objets sur lesquels doit s’exécuter la commande.

Syntaxe d'une ligne de commande

- Exemples :
    - $ ls –l /home
    - $ cal 3 2010

- Enchaînement de plusieurs commandes

    - séparation des commandes par « ; » (point-virgule)
    - exemple :
        - $ ls -l /home ; cal 3 20010

Quelques commandes simples

- **date** : affiche la date et l'heure.
- **cal** : affiche le calendrier.
- **who** : affiche la liste des utilisateurs.
- **ls** : donne la liste des fichiers dans le répertoire courant.
- **uname** : affiche le nom et les caractéristiques du système.
- **passwd** : modifie son mot de passe.

Aide du système : la commande *Man*

- **Name** : décrit en quelques mots l'objet de la documentation.
- **Synopsis** : présente la syntaxe de la commande.
- **Description** : donne la description précise du fonctionnement de la commande et de chacune de ses options.
- **Files** : fournit la liste des fichiers de configuration ou de données que la commande va mettre en œuvre.
- **See also** : renvoie à des éléments connexes de la documentation.
- **Exit status** : renseigne les codes de retour d'exécution, information utilisée pour la programmation Unix.

Autres sources de documentation

- La commande **whatis**
  - recherche un mot complet dans la base de données « whatis »
  - exemple : $ whatis date
- La commande **apropos**
  - recherche une chaîne de caractères dans la base de données « whatis ». Elle est donc moins restrictive que la commande « whatis »
  - exemple : $ apropos date

Le système de fichiers

- Un système de fichiers est une façon d'organiser et de stocker une arborescence sur un support (disque, disquette, cd ...) :
  - Chaque OS propriétaire a développé sa propre organisation.
  - On peut faire cohabiter plusieurs systèmes dans des partitions d'un même disque.
  - Linux possède son système appelé **ext2** (ext3, ext4) mais peut en gérer d'autres. La liste en est donnée dans /proc/filesystems
- Avec le système Unix, pour l'utilisateur tout est un fichier. Tous les systèmes de fichiers quels que soient leur emplacement physique doivent être intégrés dans l'UNIQUE arborescence logique du système Linux.

Les types de fichiers

- fichiers **ordinaires** :
  - texte : courrier, sources des programmes, scripts, configuration ...
  - exécutables : programmes en code binaire

- fichiers **répertoires** : fichiers conteneurs qui contiennent des références à d'autres fichiers, véritable charpente de l'arborescence, ils permettent d'organiser les fichiers par catégories

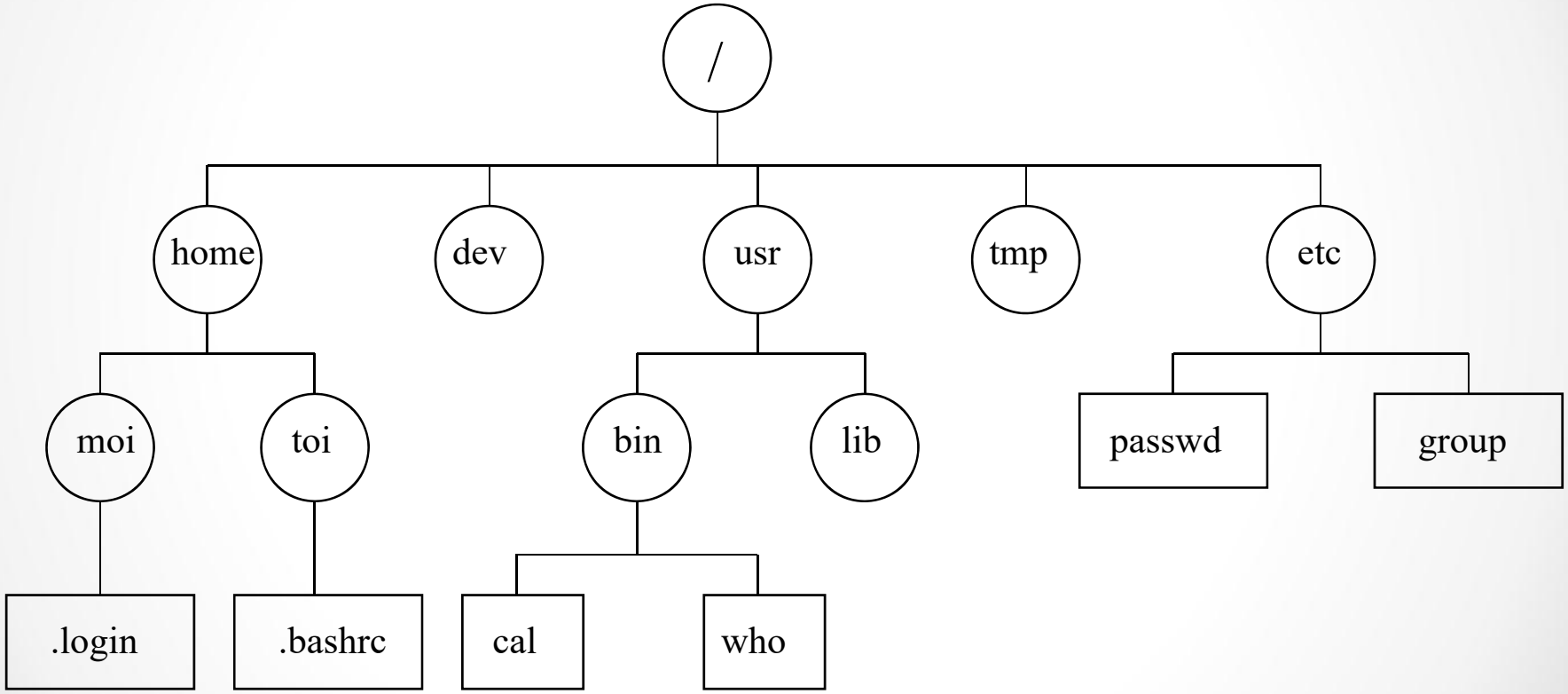
- fichiers **spéciaux** : situés dans **/dev**, ce sont les points d'accès préparés par le système aux périphériques. Le montage va réaliser une correspondance de ces fichiers spéciaux vers leur répertoire "point de montage".
  par exemple, le fichier /dev/hda permet l'accès et le chargement du 1er disque IDE

- fichiers **liens symboliques** : Ce sont des fichiers qui ne contiennent qu'une référence (un pointeur) à un autre fichier.
  Cela permet d'utiliser un même fichier sous plusieurs noms sans avoir à le dupliquer sur le disque.

Quelques répertoires

/bin	les binaires (exécutables) vitaux pour l'utilisateur
/dev	tous les périphériques
/home	les répertoires personnels des utilisateurs
/mnt	les dossiers pour les points de montage temporaires
/proc	accès direct aux paramètres du noyau
/root	répertoire personnel du super utilisateur (root)
/var	les fichiers qui changent fréquemment, tels que : les logs, les mails, les queues d'impression

Exemple de système de fichiers sous Unix



Sous Linux, toute l'arborescence part de « / », appelé « root » ou racine. Dans ce répertoire racine se trouvent tous les dossiers élémentaires

Les chemins

- Le chemin d'**accès absolu** décrit l'itinéraire à emprunter depuis la racine de l'arborescence jusqu'au fichier.
- Le chemin d'**accès relatif** décrit l'itinéraire à emprunter depuis le répertoire de travail courant. Si le fichier à atteindre est situé au dessous du répertoire de travail, l'utilisateur doit mentionner le chemin qu'il reste à parcourir.
- Le symbole « **.** » désigne le répertoire courant et le symbole « **..** » sont père
- Attention : le symbole « . » devant un nom de fichier signifie qu'il est caché.

Exemple de chemins

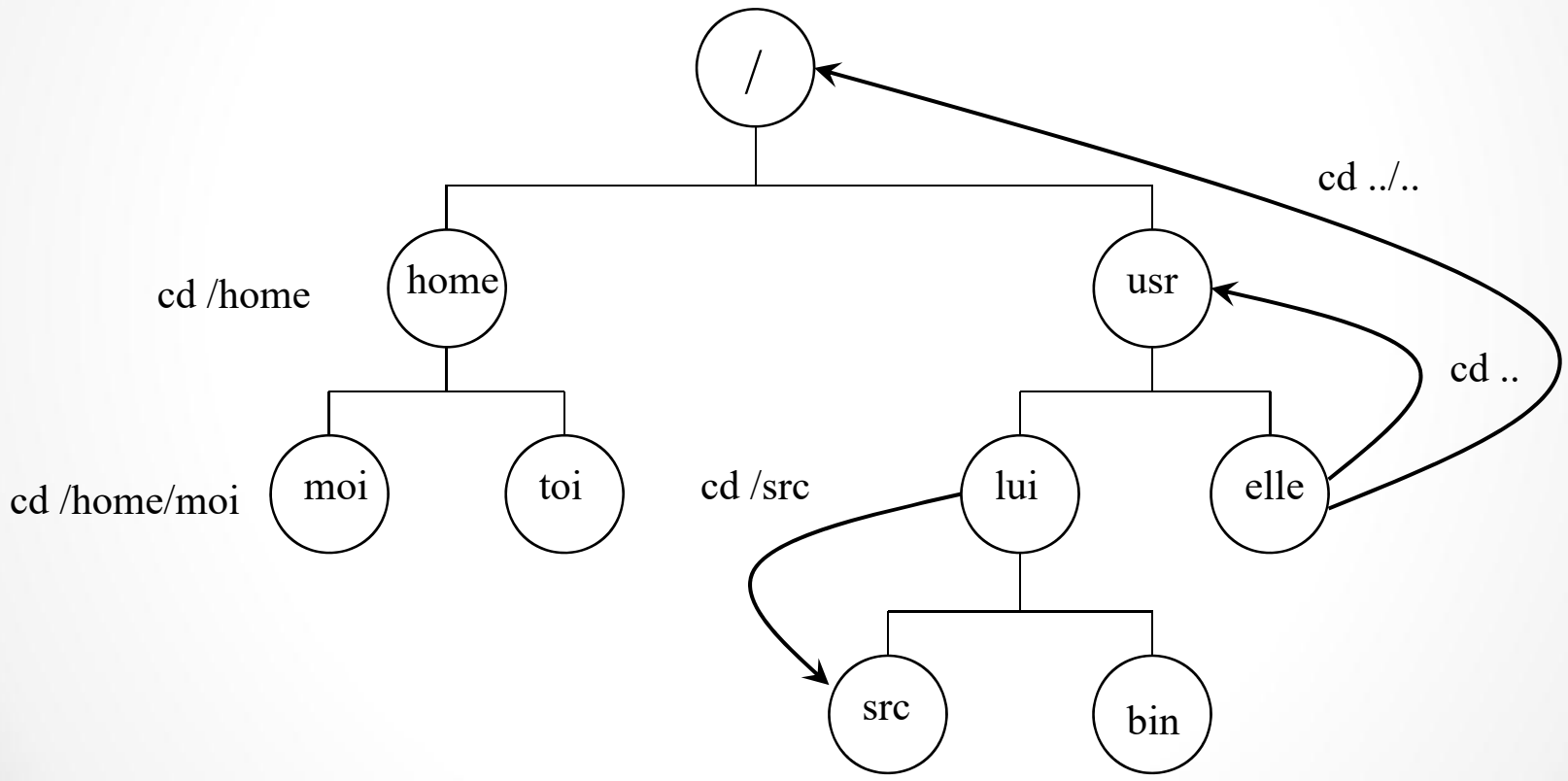
- Par exemple **/usr/bin/cal** est le chemin absolu du fichier ***cal***.

- Si le répertoire de travail est le répertoire **/home/moi** :

  - **.login** est équivalent au chemin absolu **/home/moi/.login**
  - **./.login** est équivalent au chemin absolu **/home/moi/.login**
  - **../toi/.cshrc** est équivalent au chemin absolu **/home/toi/.cshrc**

NB : Le symbole «~» désigne le répertoire de connexion de l'utilisateur. Par exemple, si l'utilisateur est **moi**, alors **~/.login** correspond au chemin relatif **/home/moi/.login**

La commande cd



Les attributs des fichiers

- $ ls –l fichier

  -rwxr-x--- 1 moi etu 300 Jun 02 12:00 fichier

propriétaire

groupe

nom

type

Droits

Nombre de liens

taille en octets

date et heure de dernière modification

- : fichier

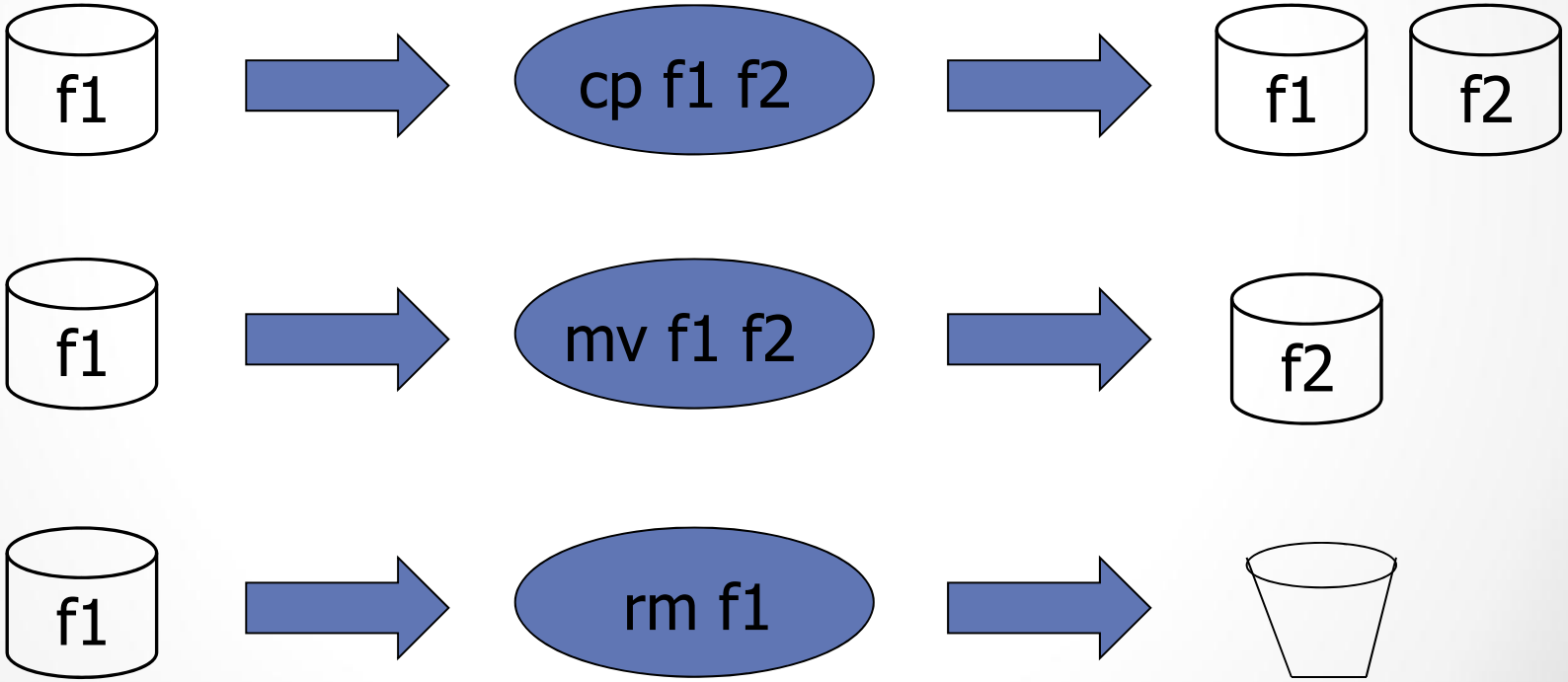
d : répertoire

l : lien

Quelques commandes pour les fichiers

- **cp** : copie d'un fichier.
- **rm** : détruit un fichier.
- **mv** : change le nom d'un fichier, déplace un fichier.
- **cat, more** : affiche le contenu d'un fichier .
- **file** : affiche le type du fichier.
- **cmp, comm, diff** : compare des fichiers.

Copier, détruire, renommer un fichier



Quelques commandes pour les répertoires

- **pwd** : affiche le répertoire courant.
- **mkdir** : crée un répertoire.
- **rmdir** : supprime un répertoire.
- **du** : affiche la taille d'une arborescence.
- **find** : recherche des fichiers dans une arborescence.

Connaître les droits

propriétaire *groupe*

- $ ls –l fichier

  -|rwx|r-x|---| 1 moi etu 300 Jun 02 12:00 fichier

Droits du propriétaire

Droits du groupe

Droits pour les autres

Droits sur les fichiers et répertoires

- **Notation symbolique**
  - Qui ?
    - U = user (propriétaire)
    - G = group (le groupe)
    - O = others (les autres)
  - Permission
    - R = read (consulter le contenu)
    - W= write (modifier et détruire)
    - X = execute (éxecuter un script ou un programme)

Droits sur les fichiers et répertoires

- **Notation octale**

  - u, g et o sont remplacés par des valeurs octales selon la règle suivante :
    - 0 = aucun droit
    - 1 = exécution (x)
    - 2 = écriture (w)
    - 4 = lecture (r)

  - exemple : $ chmod 400 fichier

Droits sur les fichiers et répertoires

$ chmod u-w fichier

u	+	*r*
g	-	*w*
ug	=	*rw*
uo		*rx*
go		*wx*
ugo		*rwx*
a		-

$ chmod 400 fichier

Droit de lecture pour le propriétaire

Droits sur les fichiers et répertoires : exemples

- **$ chmod 751 exemple.c**

  -rwxr-x--x 1 retraint users 1829 Mar 18 2002 exemple.c

  rwx = 4+2+1 = 7 (utilisateur)
  r-x = 4+1 = 5 (groupe)
  --x = 1 (autres)

$ chmod 700 ~

Permet de transformer son répertoire de connexion en répertoire privé

Liens physiques et symboliques

- La commande « ln » permet de créer un lien pointant vers un fichier (à la manière des raccourcis de Windows).
- Le lien apparaît lorsque l 'on passe la commande « ls -l ».
- Sans l 'option « -s », « ln » crée un lien physique (hard link) au lieu d 'un lien symbolique.
- Les liens symboliques permettent de ne pas dupliquer l 'espace disque utilisé par les fichiers.
- Pour supprimer un lien, on utilise la commande « rm ». C 'est bien le lien et non le fichier pointé qui est détruit.
    - $ ln -s **fichier_ou_répertoire** *lien*

Liens physiques et symboliques : exemple

- $ ln -s fich fich2

  $ ls - l

-rw - r- -r- - 1 retraint users 1829 Mar 5 14:25 fich

lrwxrwxrwx 1 retraint users 4 Mar 9 14:03 fich2 ->fich

La dernière ligne indique le lien symbolique entre les fichiers fich et fich2

- $ rm fich
- cat fich2

  Cat : fich2 : No such file or directory

Les redirections des entrées sorties

Il existe des méthodes pour rediriger les entrées et
les sorties des commandes

- Les redirections standard
  - le signe > redirige la sortie vers le fichier dont le nom suit (et le remplace)
  - le signe >> redirige la sortie vers le fichier dont le nom suit (ajout à la fin du fichier)
  - < redirige l'entrée vers le fichier dont le non suit
  - << permet de donner les arguments à une commande jusqu'au marqueur désigné

Les redirections des entrées sorties

- Rediriger l'entrée standard (le clavier)
  $ commande [options] [arguments] < fichier
  Les données qui alimentent la commande viennent du fichier

- Rediriger la sortie standard (l'écran)
  $ commande [options] [arguments] > fichier
  les résultats produits par la commande sont écrits dans le fichier

Les redirections : exemples

```
$ cat fich1 >fich2
$ cat <<FIN >sortie
> Bonjour,
> Comment ça va ?
> Au revoir.
> FIN
$ cat sortie
Bonjour,
Comment ça va ?
Au revoir.
```

Les redirections des erreurs

- Les commande envoient les lignes de texte qui ont valeur de résultat sur la sortie standard et les messages qui signalent une erreur de syntaxe ou d'exécution sur la sortie des standard des erreurs.

- Pour rediriger la sortie standard des erreurs produites par une commande, il faut mentionner dans la ligne de commande 2>fichier_de_redirection

  $ ls –l 2>/dev/null

- Pour ignorer les messages d'erreur qui parasite la lisibilité des résultats on les redirige vers le périphérique /dev/null. Elle sont perdues

Les tubes

Le tube, exprimé par le symbole |, est un mécanisme de communication qui permet d'échanger des données entre commandes.

commande1 | commande2 | commande3

$ ls –l | more : affiche le résultat de la commande ls, page par page

$ ls –l | lp : imprime le résultat de la commande ls

Liens physiques et symboliques

- La commande « ln » permet de créer un lien pointant vers un fichier (à la manière des raccourcis de Windows).
- Le lien apparaît lorsque l 'on passe la commande « ls -l ».
- Sans l 'option « -s », « ln » crée un lien physique (hard link) au lieu d 'un lien symbolique.
- Les liens symboliques permettent de ne pas dupliquer l 'espace disque utilisé par les fichiers.
- Pour supprimer un lien, on utilise la commande « rm ». C 'est bien le lien et non le fichier pointé qui est détruit.
    - $ ln -s **fichier_ou_répertoire** *lien*

Liens physiques et symboliques : exemple

- $ ln -s fich fich2

  $ ls - l

-rw - r- -r- - 1 retraint users 1829 Mar 5 14:25 fich

lrwxrwxrwx 1 retraint users 4 Mar 9 14:03 fich2 ->fich

La dernière ligne indique le lien symbolique entre les fichiers fich et fich2

- $ rm fich
- cat fich2

  Cat : fich2 : No such file or directory

Introduction aux scripts

- Le terme « script » désigne un fichier exécutable par le Shell. Il contient des commandes externes du système Unix, des commandes internes et des commentaires.
- Quand on lance l'exécution d'un script, le Shell lit et exécute séquentiellement les lignes du fichier.
- Si le shell détecte une erreur dans une ligne, il arrête la lecture du script et affiche le numéro de ligne de la commande qui a provoqué l'erreur.

Introduction aux scripts

- L'environnement d'un utilisateur (type de terminal, choix du prompt, définition des touches d'effacement des caractères,…) est construit à partir de deux scripts :
  - le script /etc/profile commun à tous les utilisateurs.
  - le script ~/.bash_profile qui se trouve dans le répertoire de connexion de l'utilisateur.

Les commentaires

- Dans une ligne de commande, le caractère « # » indique le début d'un commentaire qui se poursuit jusqu'à la fin de la ligne.

- Le pseudo commentaire « #! », placé en début de script, permet de désigner explicitement le shell qui doit exécuter le script.

- #!/bin/bash # ce script est exécuté par le Korn shell
  # Ce script affiche la liste des processus
  ps

Exécution d'un script

- Exécution d'un script
  $ bash lescript
  $ bash <lescript
  $ chmod +x ; ./lescript

- Activation/désactivation du mode trace dans le script :
  il faut inclure la commande « set –x » dans le script.

- Exécution d'un script avec l'affichage de la trace des commandes
  $ bash –xv lescript

Les paramètres

- Dans la terminologie du Shell, un *paramètre* désigne toute entité pouvant contenir une valeur. Le Shell distingue trois classes de paramètres :
  - les *variables*, identifiées par un *nom* Ex: *a* , **PATH**
  - les *paramètres de position*, identifiés par un *numéro* Ex : **0** , **1** , **12**
  - les *paramètres spéciaux*, identifiés par un *caractère spécial* Ex : **#** , **?** , $

- Le mode d'affectation d'une valeur est spécifique à chaque classe de paramètres. Suivant celle-ci, l'affectation sera effectuée par l'utilisateur, par **bash** ou bien par le système. Pour obtenir la valeur d'un paramètre, on placera toujours le caractère **$** devant sa référence, et cela quelle que soit sa classe.

- Exemple : $ echo $PATH => affiche la valeur de la variable *PATH*

Les variables

- Une variable est identifiée par un *nom*, c'est-à-dire une suite de lettres, de chiffres ou de caractères ne commençant pas par un chiffre. Les lettres majuscules et minuscules sont différenciées.

- Les variables peuvent être classées en trois groupes :
  - les variables utilisateur (ex : *a*, *valeur*)
  - les variables prédéfinies du Shell (ex : **PS1**, **PATH**, **REPLY**, **IFS**)
  - les variables prédéfinies de commandes Unix (ex : **TERM**).

- En général, les noms des variables utilisateur sont en lettres minuscules tandis que les noms des variables prédéfinies (du Shell ou de commandes Unix) sont en majuscules.

Les variables

- L'utilisateur peut affecter une valeur à une variable en utilisant :
  - l'opérateur d'affectation =
  - la commande interne **read**.
- Affectation directe :

  variable=valeur
- Valeur d'une variable :

  $variable

Exemple :

```
$ x=bon
$ echo $x
bon
$ x=$x$y
echo $x
bonjour
```

```
$ y=jour
```

Les variables

- Affectation par lecture :

  read [var1 var2 ….]

Exemple :

```
$ read x y
bon jour
echo $x$y
bonjour
```

- Lorsque la commande interne **read** est utilisée sans argument, la ligne lue est enregistrée dans la variable prédéfinie du shell **REPLY** .

Exemple :

```
$ read
bonjour tout le monde
$ echo $REPLY
bonjour tout le monde
```

Les variables

- L'option **–p** de **read** affiche une chaîne d'appel avant d'effectuer la lecture ; la syntaxe à utiliser est : **read –p** *chaîne_d_appel* **[** *var* **…** **]**

  Exemple : $ **read -p "Entrez votre prenom : " prenom**

  Entrez votre prenom : **Eric**

  $ **echo $prenom**

  Eric

- Liste des variables :

  La commande « set » liste l'ensemble des variables

- Suppression d'une variables :

  unset variable

Définition de séparateur de champs : IFS

- La commande « read » lit une ligne entière et utilise la variable « IFS » pour décomposer la ligne en mots qu'elle va affecter aux différentes variables de lecture qui lui sont passées en argument.

- Par défaut, la variable « IFS » est initialisée avec les caractères <Espace>, <tabulation> et <Saut de ligne>.

- Modification de la variable « IFS » :
  - IFS= :

Les variables en lecture seule

- Pour définir une variable dont la valeur ne doit pas être modifiée (appelée *constante* dans d'autres langages de programmation), on utilise la syntaxe : **declare –r** *nom=valeur* [*nom=valeur* …]

- Exemple : $ **declare -r mess=bonjour**

- Une tentative de modification de la valeur d'une constante provoque une erreur.

- Exemple : $ **mess=salut**
  -bash: mess: readonly variable

- Pour connaître la liste des constantes définies : **declare –r**

Les variables d'environnement

$ var_locale=alpha
$ var_environ=gamma
$ export var_environ

- La commande « export » permet à un shell de placer une variable dans l'environnement.

  Syntaxe $ export variable[=valeur]

- La commande « env » permet de lister l'ensemble des variables exportées.

Les paramètres

- Un script est à l'image d'une commande. Il est possible de fournir, sur la ligne de commandes, les arguments nécessaires à son exécution.
- Les arguments sont affectés aux variables 0,1,…,9,10,11,…
- La variable « * » désigne l'ensemble des paramètres sous la forme d'un seul argument.
- La variable « @ » désigne l'ensemble des paramètres, un argument par paramètre

Les paramètres

- La variable « # » désigne le nombre de paramètres passés au script.

- La commande « shift » permet de décaler les paramètres : la valeur de la variable 1 est remplacer par celle de la variable 2, celle de la variable 2 par celle de la variable 3, etc.

  shift [n] : décale de n paramètres

- La commande interne « set » permet de remplacer tous les paramètres.

```
$ set un deux trois ; echo $*
un deux trois
```

Suppression des ambigüités

- Pour éviter les ambiguïtés dans l'interprétation des références de paramètres, on utilise la syntaxe **${*paramètre*}**.

- Exemples :

  $ **set un deux trois quatre cinq six sept huit neuf dix onze douze**
  $ **echo $11**
  un1

  $ **echo ${11}** => pour obtenir la valeur du onzième paramètre de position
  onze

Substitution de commandes

- Une commande *cmd* entourée par une paire de parenthèses **( )** précédées d'un caractère **$** est exécutée par le shell puis la chaîne **$(** *cmd* **)** est remplacée par les résultats de la commande *cmd* écrits sur la sortie standard. Ces résultats peuvent alors être affectés à une variable ou bien servir à initialiser des paramètres de position.

- Exemple : $ **pwd**
  /home/retraint
  $ **repert=$(pwd)** => la chaîne */home/retraint* remplace la chaîne *$(pwd)*
  $ **echo mon repertoire est $repert**
  mon repertoire est /home/retraint

Substitution de commandes

- Lorsque la substitution de commande est utilisée avec la commande interne **set**, l'erreur suivante peut se produire :

  $ **set $(ls -l .bashrc)**

  bash: set: -w: invalid option

  $ **ls -l .bashrc**

  -rw-r--r-- 1 retraint users 1342 nov 30 2009 .bashrc

- Le premier mot issu de la substitution commence par un caractère **tiret** : la commande interne **set** l'interprète comme une option, ce qui provoque une erreur. Pour résoudre ce type de problème, on double le caractère **tiret**.

  Exemple : $ **set -- $(ls -l .bashrc)**

  $ **echo $1**

  -rw-r--r—

Substitution de commandes

- Les substitutions de commandes peuvent être imbriquées.

  Exemple :

  $ **set $(ls $(pwd)/retraint)**

  $ **echo $#** => nombre d'entrées du répertoire retraint

  3

- La substitution de commande permet également de capter les résultats écrits par un fichier shell.

  Exemple : $ lescript

  38

  $ **resultat=$(lescript)**

  La variable *resultat* contient la chaîne 38

Le code retour

- Une commande Unix ou un script qui se termine émettent un code de retour numérique.

- La variable prédéfinie « ? » est automatiquement initialisée avec le code retour de la dernière commande exécutée.

```
$ who | grep thomas
$ echo $?    # thomas est-il connecté ?
0            # thomas est connecté
```

Les instructions de contrôle

- Comme tous les langages de programmation, le langage shell possède des instructions de contrôle telles que par exemples:
  - L'alternative : l'instruction if.
  - Les structures itératives : les instructions for, while, until.
  - Le choix multiple : l'instruction case.

if : l'alternative

- **if** condition

  then

  bloc de commandes

  elif condition

  then

  bloc de commandes

  else bloc de commandes

  fi

L’alternative : exemple

```
If test -f $1
        then
                file $1
        else    echo ‘’ le fichier $1 n’existe pas ”
fi
```

- Si le fichier dont le nom est passé en paramètre existe, alors il affiche le type de son contenu.
- Il est possible de positionner les mots clés « if » et « then » sur la même ligne en les séparant par un point virgule :
    - if test -f $1 ; then file $1

L’alternative && et ||

- Les opérateurs « && » et « || » permettent de conditionner l’exécution d’une commande par le code retour de celle qui précède.

- commande1 && commande2
  - équivaut à : if commande1 ; then commande2 ; fi
  - « commande2 » est exécutée si « commande1 » a renvoyée 0.

- commande1 || commande2
  - Équivaut à : if commande1 ; then ; else commande2 ; fi
  - « commande2 » est exécutée si « commande1 » a renvoyée 1.

La commande test

- La syntaxe
  - if **test** expression ; then ….. fi
  - if **[** expression **]** ; then ….. fi
- La commande test n'affiche pas de résultat sur la sortie standard, elle renvoie le code retour 0 si l'expression est vraie et 1 si elle est fausse.

Test d’un attribut de fichier

- **-f fichier** : vrai si le fichier existe et qu’il est ordinaire.
- **-e fichier** : vrai si le fichier existe.
- **-r fichier** : vrai si le fichier existe et qu’il est accessible en lecture.
- **-w fichier** : vrai si le fichier existe et qu’il est accessible en écriture.

Test sur des chaînes de caractères

- **-z chaîne** : vrai si la longueur de la chaîne est 0.
- **-n chaîne** : vrai si la longueur de la chaîne est différente de 0.
- **chaîne1 = châine2** : vrai si chaîne1est identique à châine2
- **chaîne1 != châine2** : vrai si chaîne1 est différente de châine2

Test sur les nombres

- **arg1 –eq arg2** : vrai si arg1 est égal à arg2.
- **arg1 –ne arg2** : vrai si arg1 est différent de arg2.
- **arg1 –lt arg2** : vrai si arg1 est strictement inférieur à arg2.
- **arg1 –gt arg2** : vrai si arg1 est strictement supérieur à arg2.
- « **le** » et « **ge** » pour inférieur ou égal et supérieur ou égal.

Opérateurs logiques

- **!expression_logique** : vrai si expression_logique est fausse.
- **expression_logique1 –a expression_logique2** : vrai si les deux expressions sont vraies.
- **expression_logique1 –o expression_logique2** : vrai si l'une des deux expressions est vraie.

La structure case

- **case** paramètre **in**
  choix1.1 | choix 1.2)
  commande1**;commande2;;**
  [choix2) commandes **;;**]
  [*) commandes **;;]**
  esac

- Chaque valeur du paramètre est terminée par «) » . Si plusieurs valeurs sont à traiter de la même manière, on les sépare par le signe « | ».

- Chaque bloc de commandes se termine par « ;; ». La choix par défaut se marque par « *) » et est toujours situé en dernier.

La structure case : exemple

```
$ cat supprime

    echo “voulez vous supprimer le fichier ?“
    read reponse
    case $reponse in
                O | o  )  rm $1 ;;
                N | n  )  echo “le fichier n’est pas supprimer ;;
                * )       echo “réponse incorrecte“ ;;
     esac
```

La boucle tant que

- Syntaxe :

```
while  condition
do
        commandes
done
```

- Exemple :

```
while [ -r $1 ]
do
        cat $1 >> result
        shift
done
```

- Concatène dans le fichier « result » l'ensemble des fichiers dont les noms sont donnés en argument.

La boucle répéter jusqu 'à

- Syntaxe :

```
until  condition
do
        commandes
done
```

- Exemple :

```
until [ ! -r $1 ]
do
        cat $1 >> concat
        shift
done
```

- Notez la négation du test grâce à l'opérateur de négation.

La boucle for

- Syntaxe : **for** paramètre [**in** liste]
  do
  commandes
  done
- Exemple : **for i** in `ls` (for i in $(ls))
  do
  cp $i /tmp/$i
  echo " $i copié"
  done
- « 'ls' » signifie le résultat de l 'exécution de « ls ». Recopie chaque fichier du répertoire courant dans « /tmp ».

Les sauts inconditionnels

```
while   …    ;  do
        ….
        if … ; then
                continue
        fi
        if … ; then
                break
        fi
done
…..
```

L’arithmétique

- Pour définir et initialiser une ou plusieurs variables de type entier, on utilise la syntaxe suivante :

  declare -i *nom*[=*expr_arith*] [*nom*[=*expr_arith*] ...]

  Exemples :

  $ **declare -i a=1**

  $ **declare -i a b**

  $ **read a**

- Il n'est pas nécessaire de définir une variable avant de l'utiliser.

La commande interne (( ))

- Cette commande interne est également utilisée pour effectuer des opérations arithmétiques.

  - Syntaxe : **((** *expr_arith* **))**

- Exemples :

```
$ set $(date)
$ declare –i a=0
$ ((a=$2+1))
$ echo $a
$ 11
```

La commande interne (( ))

- Pour obtenir la valeur de l'expression arithmétique, on utilise la syntaxe : ***$((expr_arith))***
- Exemple : echo **$((** 7 * 3 **))**
- Substitutions de commandes et substitutions de paramètres peuvent être présentes dans une commande interne **(( ))** ou bien dans une substitution d'expression arithmétique **$(( ))** si le résultat aboutit à un nombre entier.
- Par exemple, dans l'expression **$(($(ls -l | wc -l) -1))** le résultat du pipeline **ls -l | wc –l** est interprété comme un nombre.

Les opérateurs

- On retrouve l'ensemble des opérateurs du langage C :

  a++ et a-- : post-incrémentation, post-décrémentation

  ++a et --a : pré-incrémentation, pré-décrémentation

  -a et +a : moins unaire, plus unaire

  a**b : exponentiation a^b

  a*b, a/b et a%b : multiplication, division entière, reste

Structure for pour les expressions arithmétiques

- Bash a introduit une nouvelle structure **for** adaptée aux traitements des expressions arithmétiques, itération issue du langage C.

- Syntaxe :

```
for (( expr_arith1 ; expr_arith2 ; expr_arith3 ))
do
        suite_cmd
done
```

- Exemple :

```
declare -i x y
for (( x=1,y=10 ; x<4 ; x++,y-- ))
do
        echo $(( x*y ))
done
```

Les fonctions

- **Syntaxe :**

```
function Nom  {
lignes de code ;
}
```

ou

```
Nom ( ) {
lignes de code ;
}
```

- **Déclaration des variables locales**
  - local -i var : déclaration d'un entier var
  - local –a tab : définition d'une tableau

Les fonctions

- Une fonction est exécutée en invoquant son nom suivi d'éventuels paramètres qui seront référencés de la même manière qu'un script ($0, $1, …).
- La commande « unset –f » permet de supprimer une fonction
- La commande « return » met fin à une fonction. Le code retour est stocké, comme le retour d'un script, dans la variable « ? » du shell.

Exporter une fonction

- Pour qu'une fonction puisse être exécutée par un programme Shell différent de celui où elle a été définie, il est nécessaire d'exporter cette fonction. On utilise la commande interne **export**.

  Syntaxe : **export -f *nomfct* ...**

- Pour que l'export fonctionne, le sous-shell qui exécute la fonction doit avoir une relation de descendance avec le programme Shell qui exporte la fonction.

Les tableaux

- Pour créer un tableau, on utilise généralement l'option **-a** (comme *array*) de la commande **declare** :

  declare -a *nomtab* ...

- Le tableau *nomtab* est simplement créé mais ne contient aucune valeur : le tableau est défini mais n'est pas initialisé.

- Pour connaître les tableaux définis : **declare -a**

- Pour définir et initialiser un tableau : **declare -a *nomtab*=(** *val0 val1* **...)**

- Comme en langage C, l'indice du tableau débute toujours à **0.**

Les tableaux

```
$ tableau[0]=un
$ tableau[1]=deux
$ tableau[2]=trois

$ echo ${tableau[0]}
un

$ tableau[1]=six
$ echo ${tableau[*]}
un six trois

$ echo ${#tableau[*]}
3
```